

¿Qué es DataOps?

JUAN DAVID VELASQUEZ-HENAO, MSc, PhD
Profesor Titular
Departamento de Ciencias de la Computación y la Decisión
Facultad de Minas
Universidad Nacional de Colombia, Sede Medellín

✉ jdvelasq@unal.edu.co
🐦 [@jdvelasquezh](https://twitter.com/jdvelasquezh)
🔗 <https://github.com/jdvelasq>
📄 <https://goo.gl/prkJAq>
📄 <https://goo.gl/vXH8jy>

Definición de DataOps

Herramientas y métodos que combinan data analytics, lean thinking, agile y DevOps asegurando la calidad impecable del dato:

- Prácticas ágiles para asegurar que se trabaja en las cosas correctas que adicionan valor a las personas correctas.
- Lean thinking (pensamiento esbelto) para eliminar basura y cuellos de botella, mejorar la calidad, monitorear flujos de datos y hacer más baratos los datos a los consumidores.
- DevOps establece una cultura de colaboración entre equipos históricamente aislados impulsando la eficiencia.



Puntos clave:

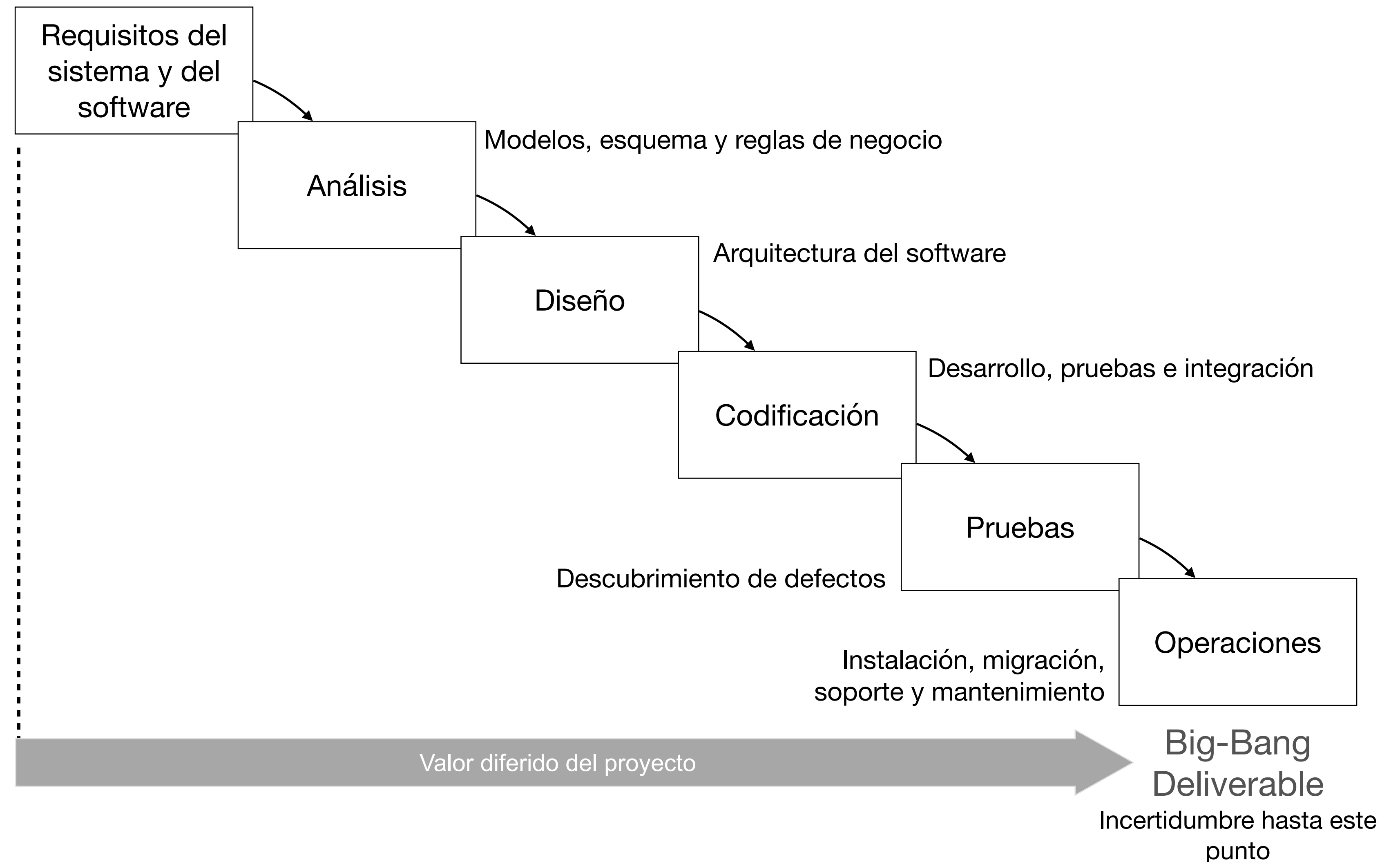
- Adapta técnicas de DevOps, Agile, Lean thinking, pero no las copia.
- No está limitado a DS/ML o Big Data y es aplicable a cualquier producto de datos.
- Es una filosofía, no una herramienta.
- Es una aproximación operativa y por si solo no da insights.

- Producto o servicio que incorpora DS.
- Resuelve un problema / da utilidad.
- Combina datos con algoritmos para inferencia, predicción u optimización.
- Es rápido, escalable y repetible.
- Es reproducible.
- Su uso es continuo
- Su monitoreo es constante
- iteración basada en la experimentación.
- Entrega retroalimentación para su mejora.

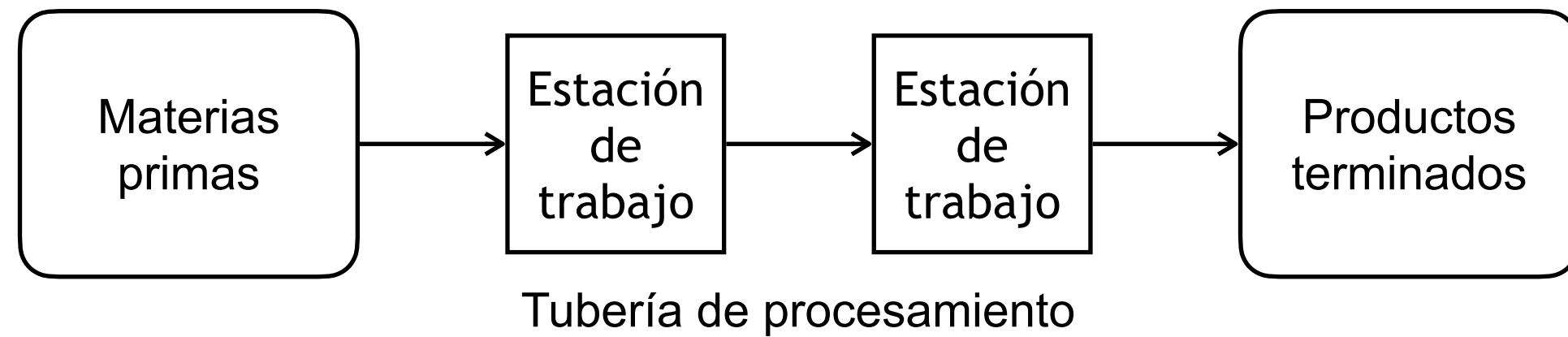
Modelo de cascada para el desarrollo de software

- Requiere planeamiento detallado y alto grado de detalle
- Funciona mejor para requerimientos fijos y bien entendidos desde el principio
- Los cambios generan ciclos de replaneamiento y causa retrasos y sobrecostos.
- Diferencia entre los ambientes de desarrollo, prueba y productivo.
- Diferencias entre el hardware en la distintas etapas.
- Falta de replicabilidad.
- Incapacidad para respuesta rapida a cambios

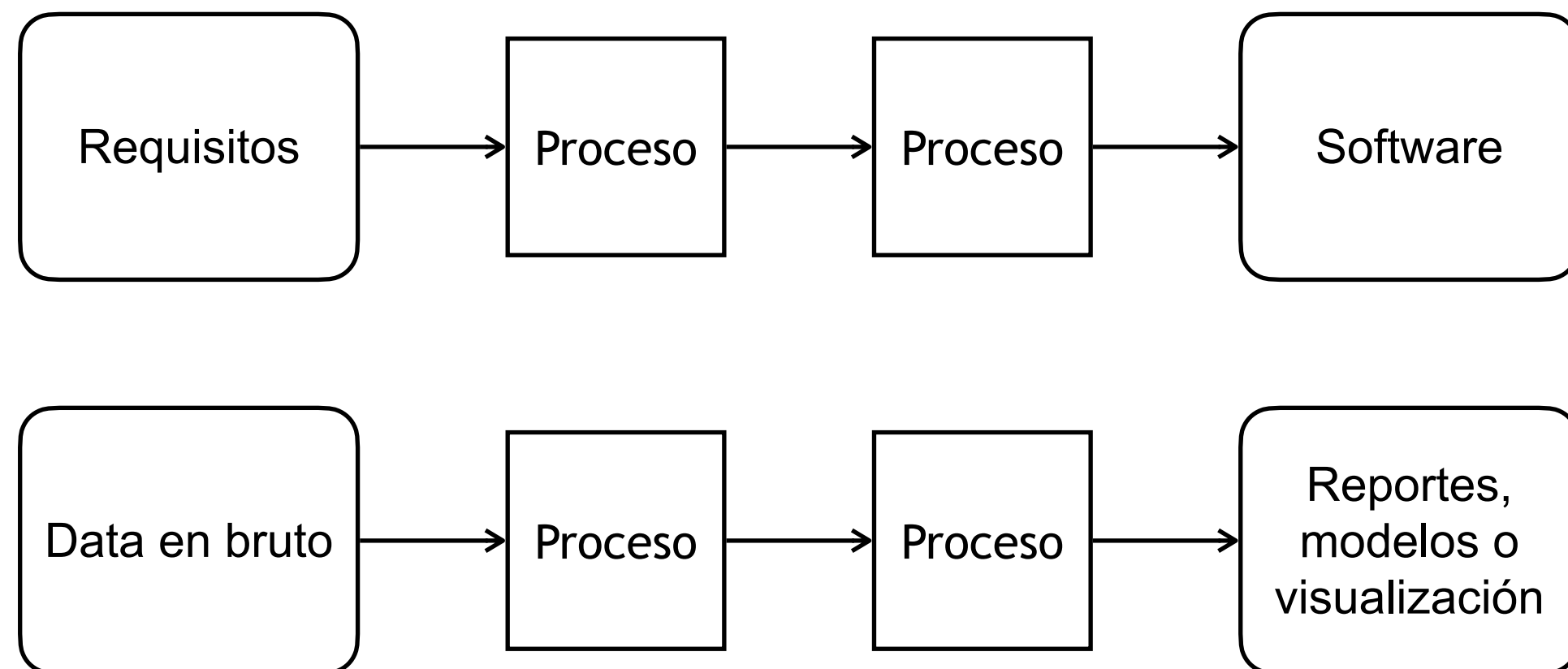
Propuesto originalmente por WW Royce en 1970, revisado por B. Boehm en 1980 y por I. Sommerville en 1985



Lean Manufacturing



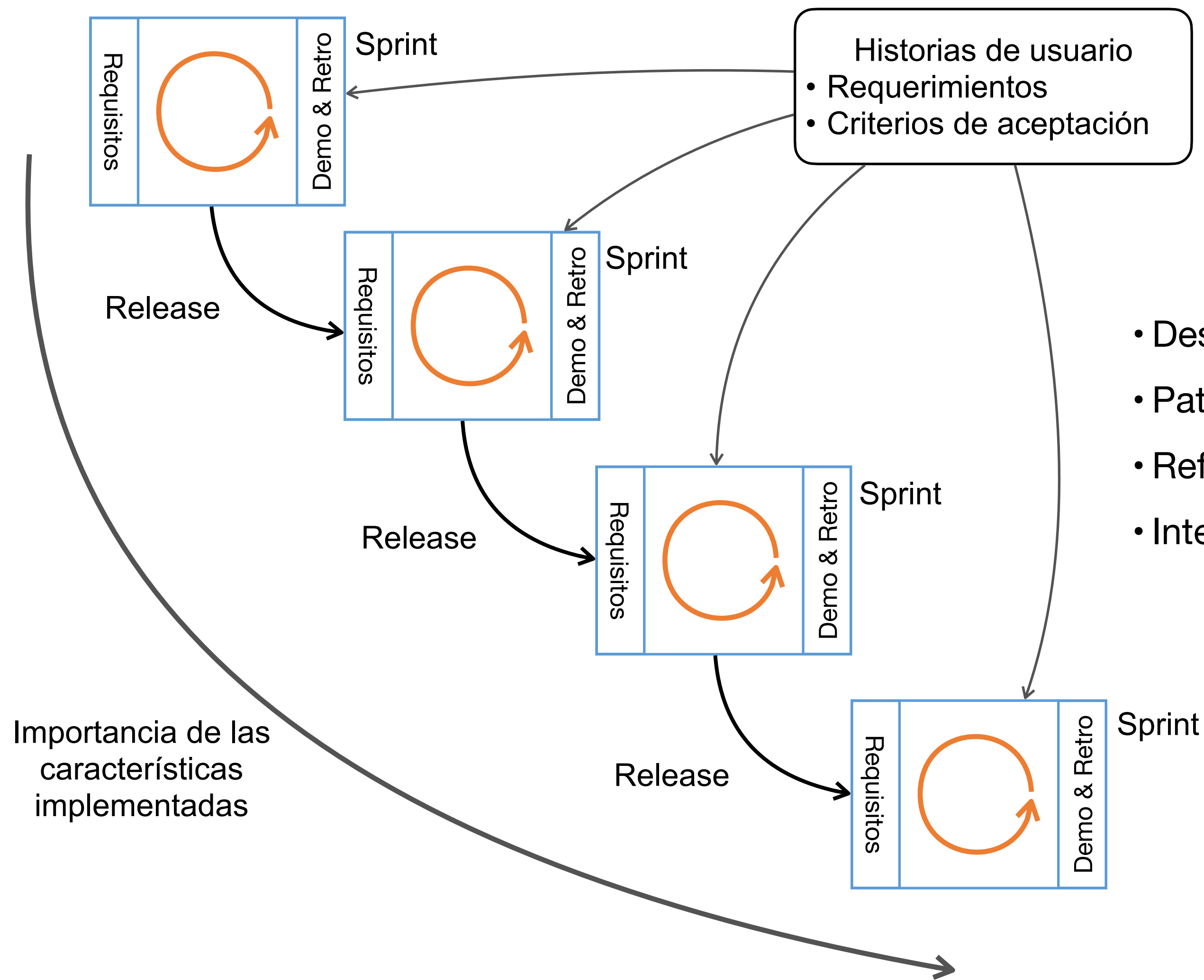
- Mejoramiento continuo de la calidad.
- Eliminación de actividades que no agregan valor.
- Cada estación de trabajo tiene una entrada bien definida, un proceso claro y una salida establecida.
- Uso de métricas de calidad (**control de procesos**).



- Tests para verificar los datos (lógica de negocio, tipo de dato, outliers, tendencias, consistencia, ...)
- Tests para verificar los modelos (precisión, degradación del modelo, ...)

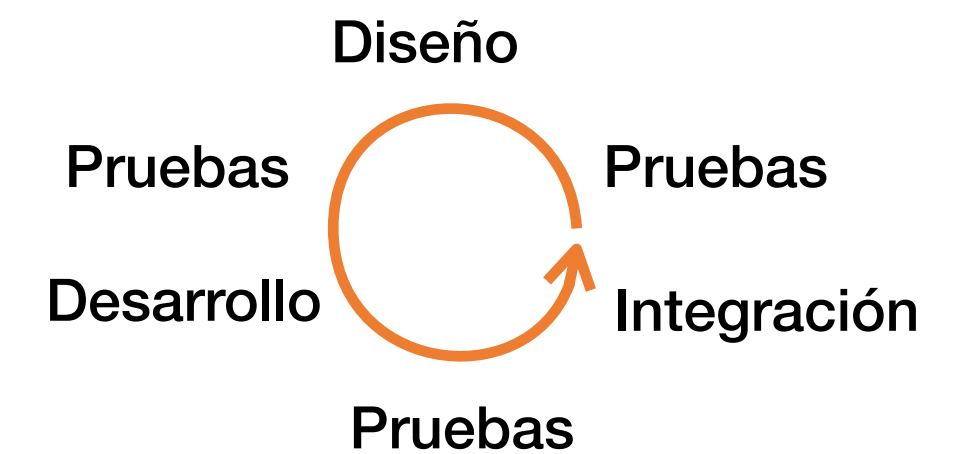
- Lógica del negocio — Valida supuestos sobre los datos. Por ejemplo:
 - Validación de clientes.
 - Validación de datos.
- Entradas — Verifica las entradas en cada paso del pipeline
 - Conteo
 - Conformidad
 - Historia
 - Balance
 - Consistencia temporal
 - Consistencia de la aplicación
 - Verificación de los campos
- Salidas — Verifica los resultados de una operación.
 - Completitud
 - Verificación de rango

Agile & Agile Manifesto



- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

- Desarrollo basado en pruebas
- Patrones de diseño
- Refactorización de código
- Integración y despliegue manual



Metodología efectiva para productos no secuenciales con requerimientos que evolucionan rápidamente

DevOps (Development + Operations)

Modelo de Cascada



Desarrollador

Puesta en Productivo



Profesional IT/Ops

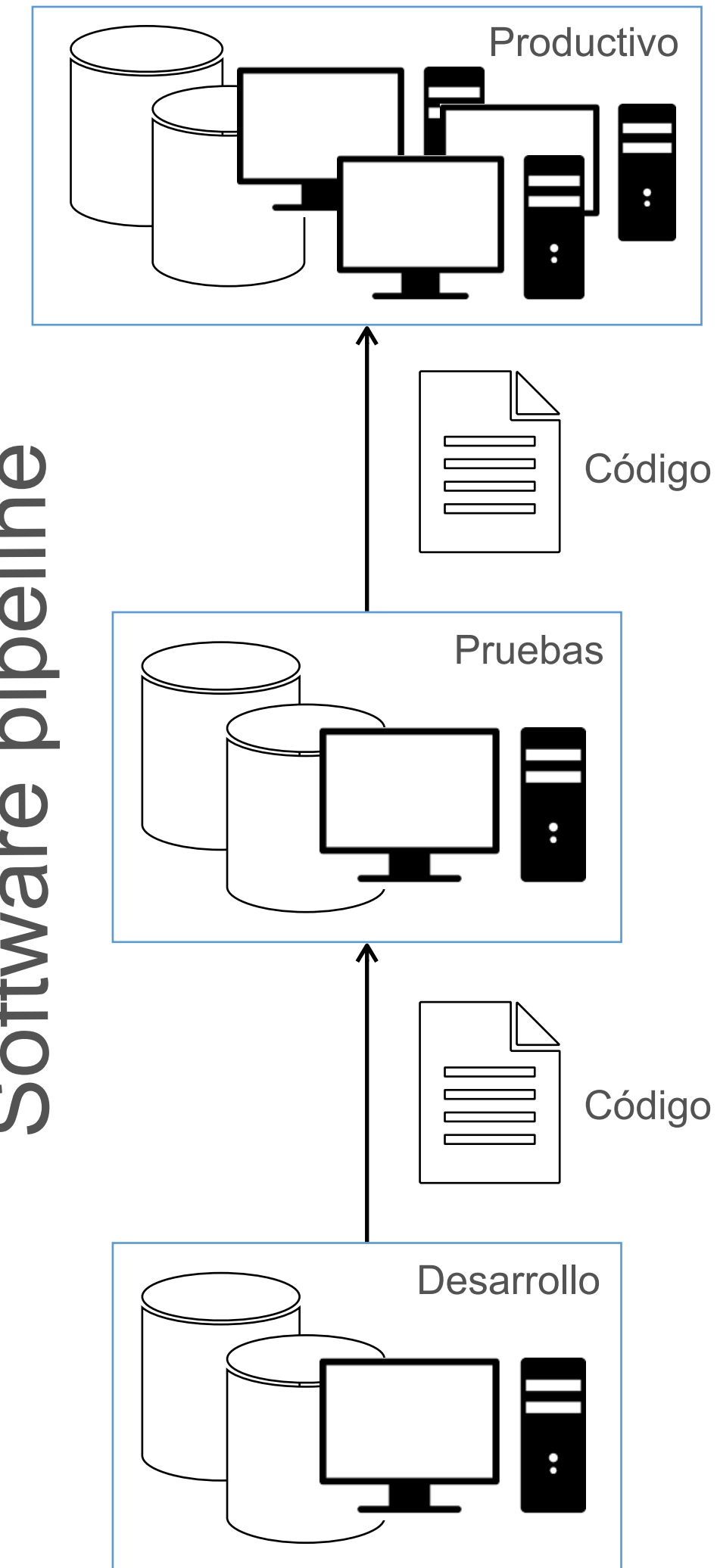
- Departamentos separados
- Métricas de éxito diferentes
- Actualización manual del codebase
- Verificación manual
- Despliegue lento del código (semanas o meses)
- Dificultad en el manejo de versiones.
- Dificultades en el manejo de incidentes
- Dificultades en el manejo de servicios IT

Esquema clásico

- Codificación
- Construcción (integración de código).
- Pruebas de software
- Empaquetado
- Versionado
- Configuración de la infraestructura
- Monitoreo

- ¿Cuánto tiempo requiere para llevar el código listo a código ejecutando en producción?
- ¿Cada cuánto libera versiones?
- ¿Cuánto tiempo requiere para restablecer el servicio?
- ¿Qué porcentaje de cambios en el software resultan en degradación del servicio que requiere intervención?

Software pipeline



- Infraestructura diferente en cada ambiente.
- Dificultad y lentitud para homogeneizar hardware y software a través de ambientes.
- Actualización constante del software base.
- Dificultad para replicar errores entre ambientes.

DevOps (Development + Operations)

Serverless Computing / Function as a Service (FaaS)

Modelo de desarrollo que permite crear y ejecutar aplicaciones y procesos sin entrar en contacto con el servidor.

Software as a Service (SaaS)

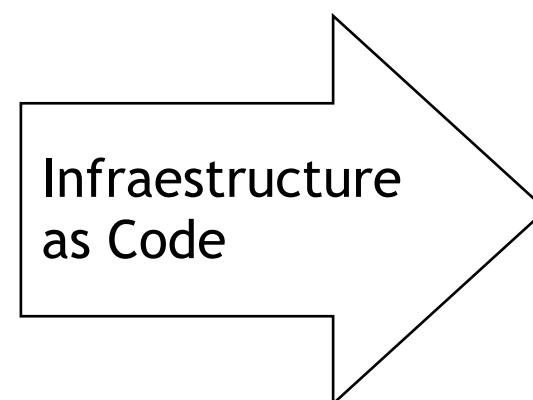
El proveedor aloja y ejecuta las aplicaciones y las hace accesibles a los clientes. El cliente solo es responsable de los datos.

Platform as a Service (PaaS)

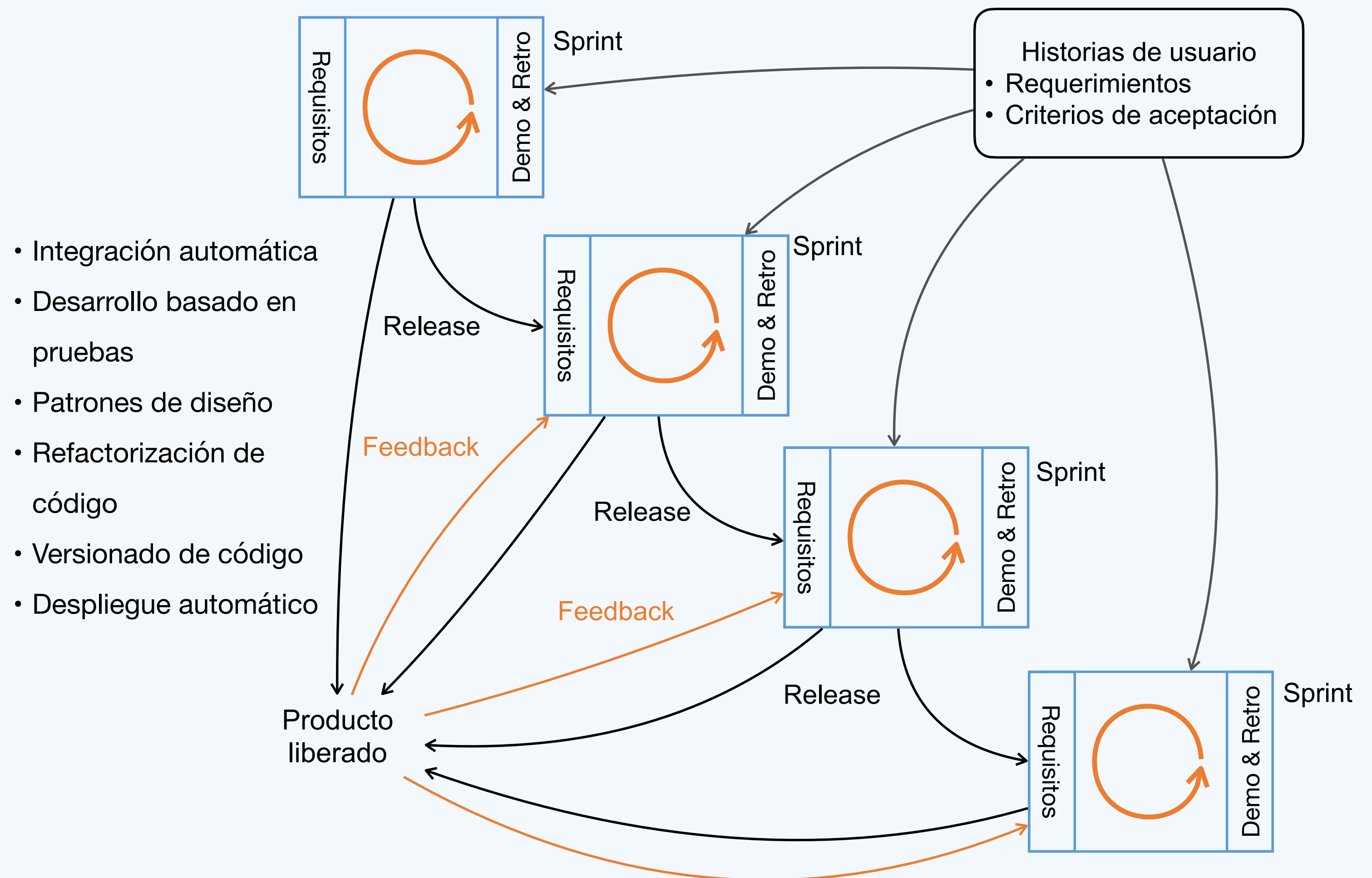
Alquiler de servidores y almacenamiento y conectividad en la nube para desarrollo, e incluye sistema operativo, lenguajes, bases de datos y demás herramientas.

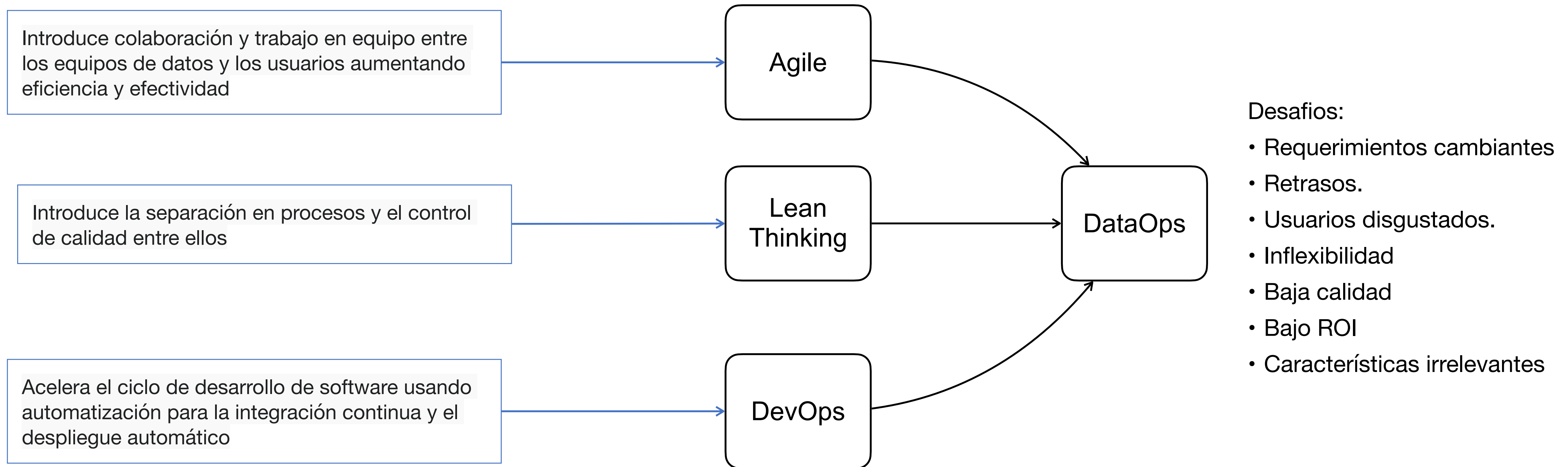
Infrastructure as a Service (IaaS)

Alquiler de servidores y almacenamiento y conectividad en la nube. El usuario es responsable de las VM, sistema operativo, soporte a datos, etc.



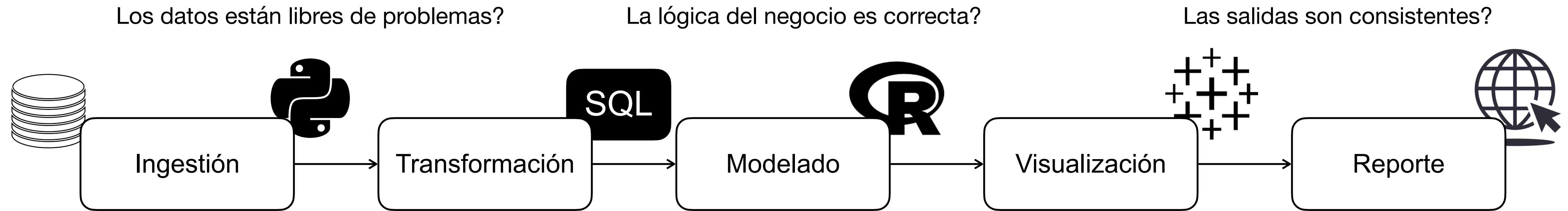
DevOps es un conjunto de prácticas destinadas a reducir el tiempo entre el compromiso de un cambio en un sistema y el cambio que se coloca en la producción normal, al tiempo que garantiza una alta calidad





Pasos para implementar DataOps

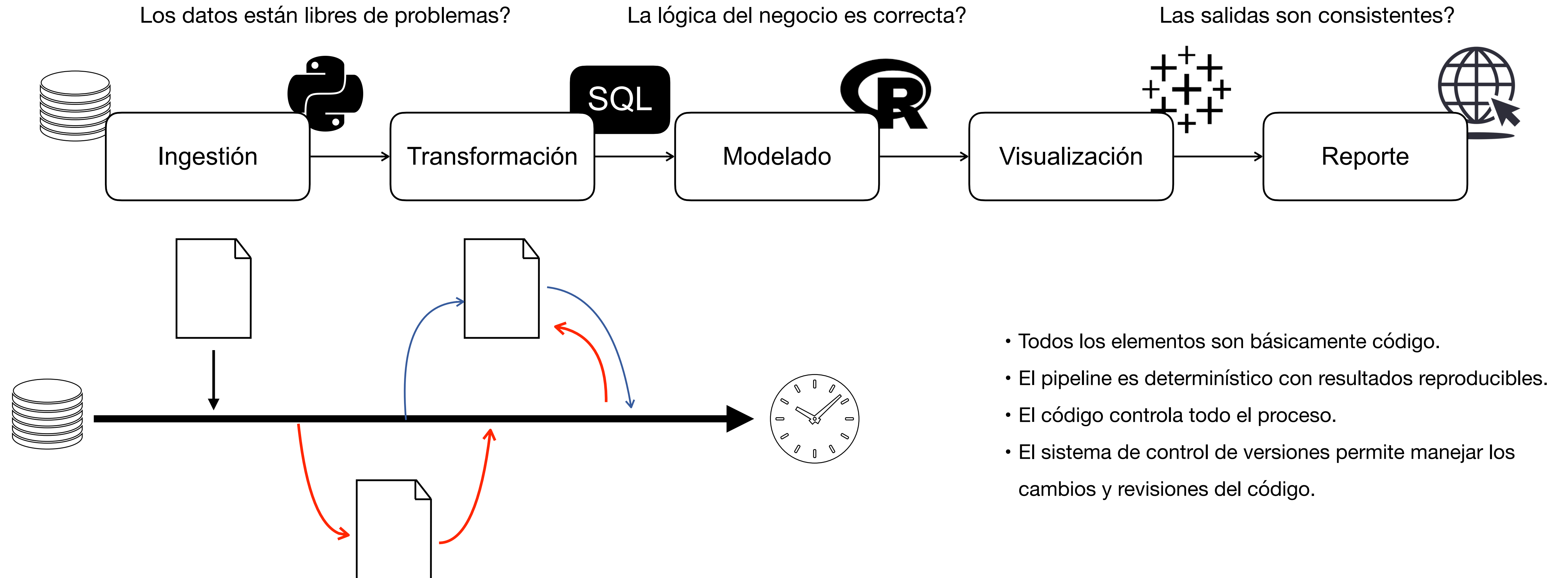
Paso 1 — Adicione pruebas de lógica y de datos



- Se verifica que cada cambio (datos, modelos y lógica) no haga fallar el sistema.
- Hay al menos un test en cada paso.
- Se inician por tests simples y se aumenta complejidad.
- Los tests también puede alertar situaciones extrañas.
- Los tests automáticos continuamente monitores el pipeline

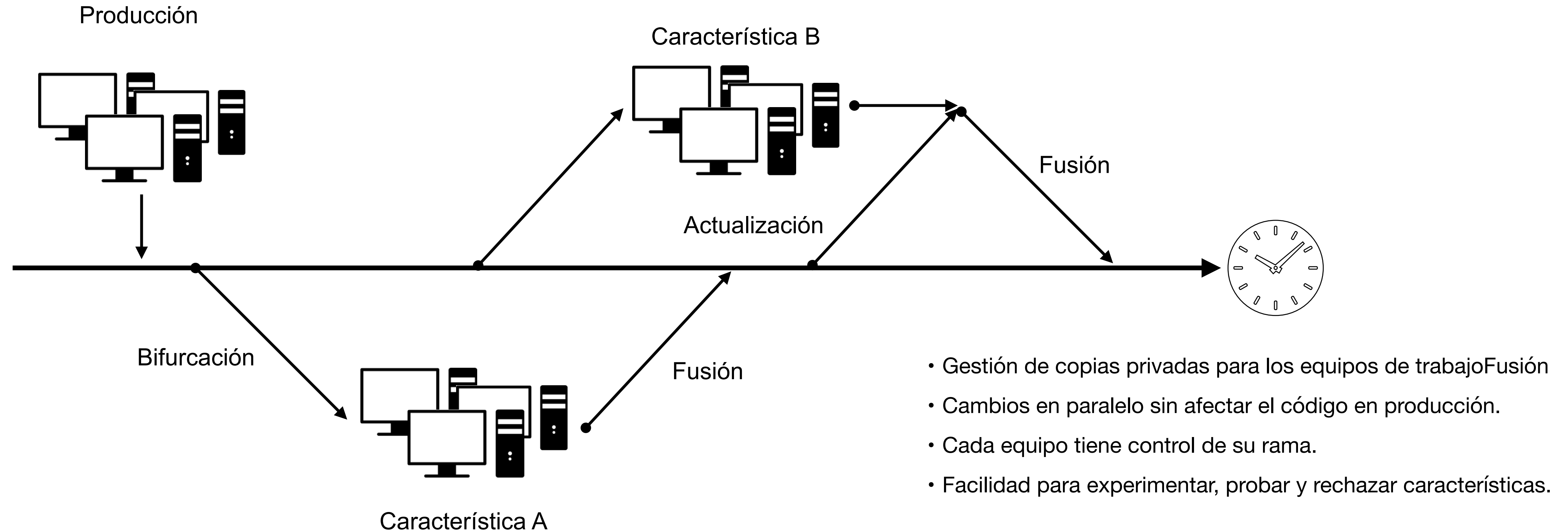
Pasos para implementar DataOps

Paso 2 — Use un sistema de control de versiones



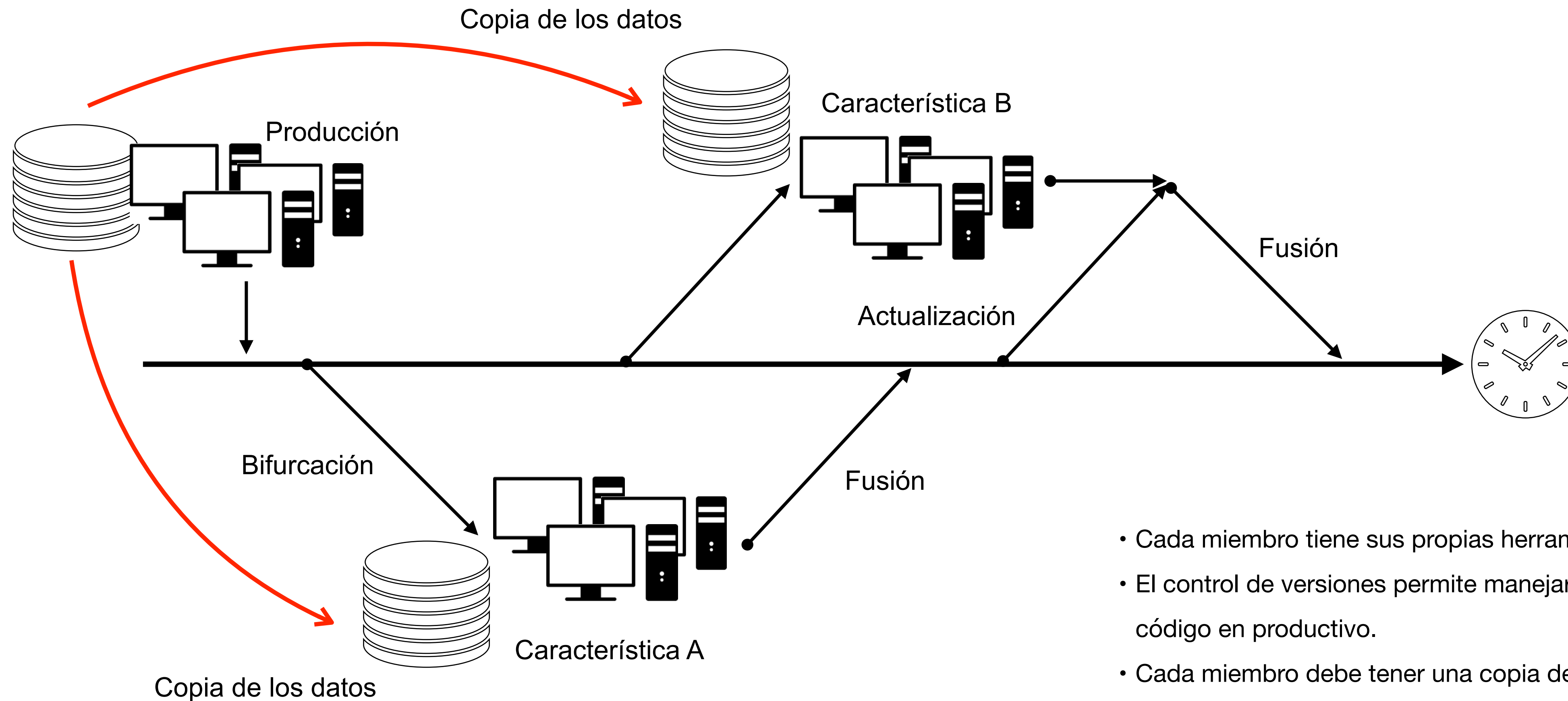
Pasos para implementar DataOps

Paso 3 — Bifurque y fusione



Pasos para implementar DataOps

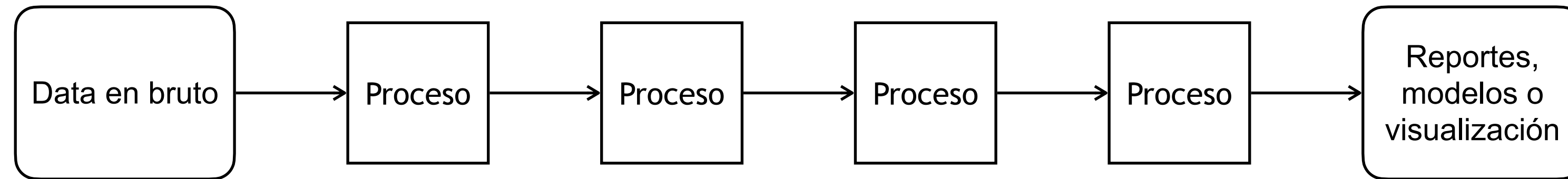
Paso 4 — Use múltiples ambientes



- Cada miembro tiene sus propias herramientas de desarrollo.
- El control de versiones permite manejar una copia privada del código en productivo.
- Cada miembro debe tener una copia de los datos que requiere.
- La separación en ambientes permite gestionar cambios en el código y en los datos

Pasos para implementar DataOps

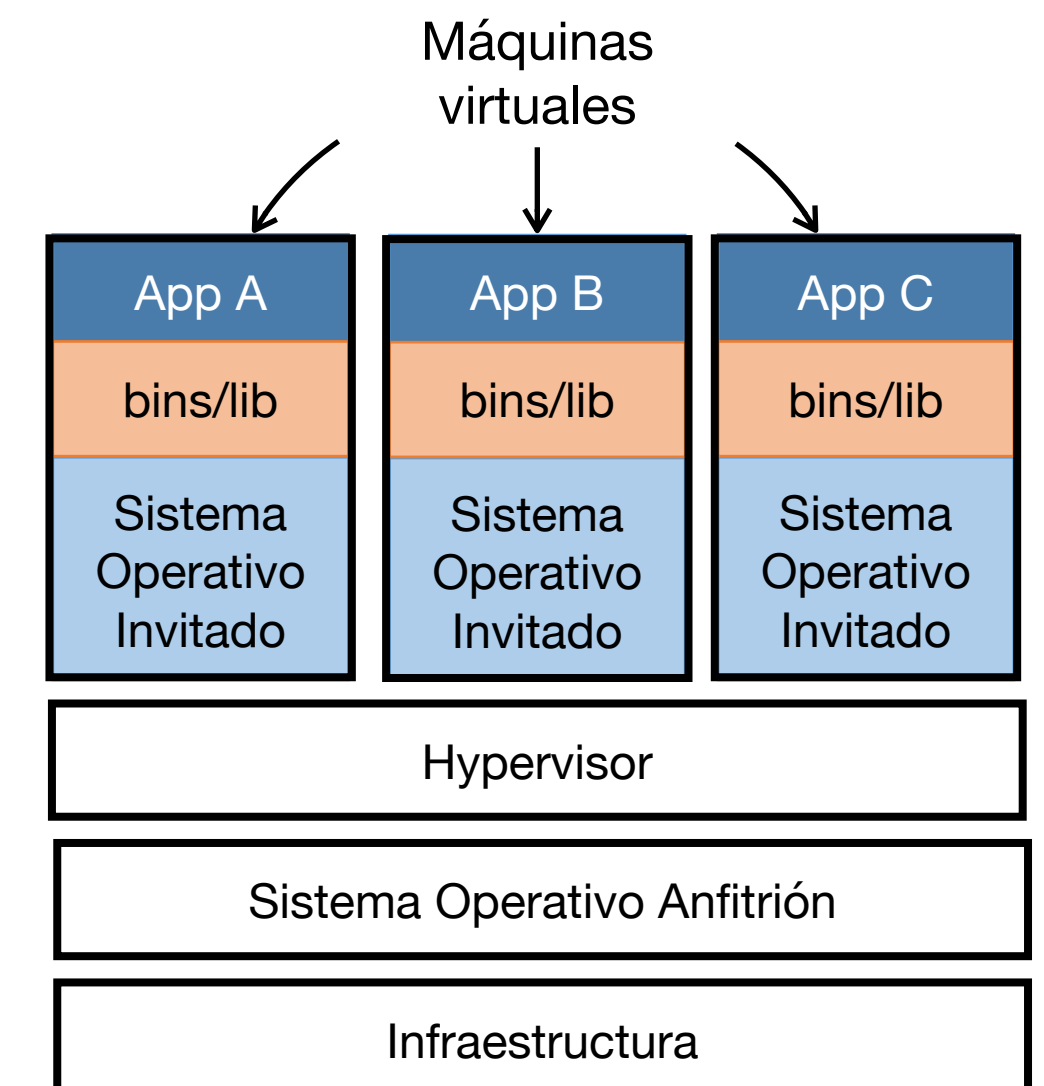
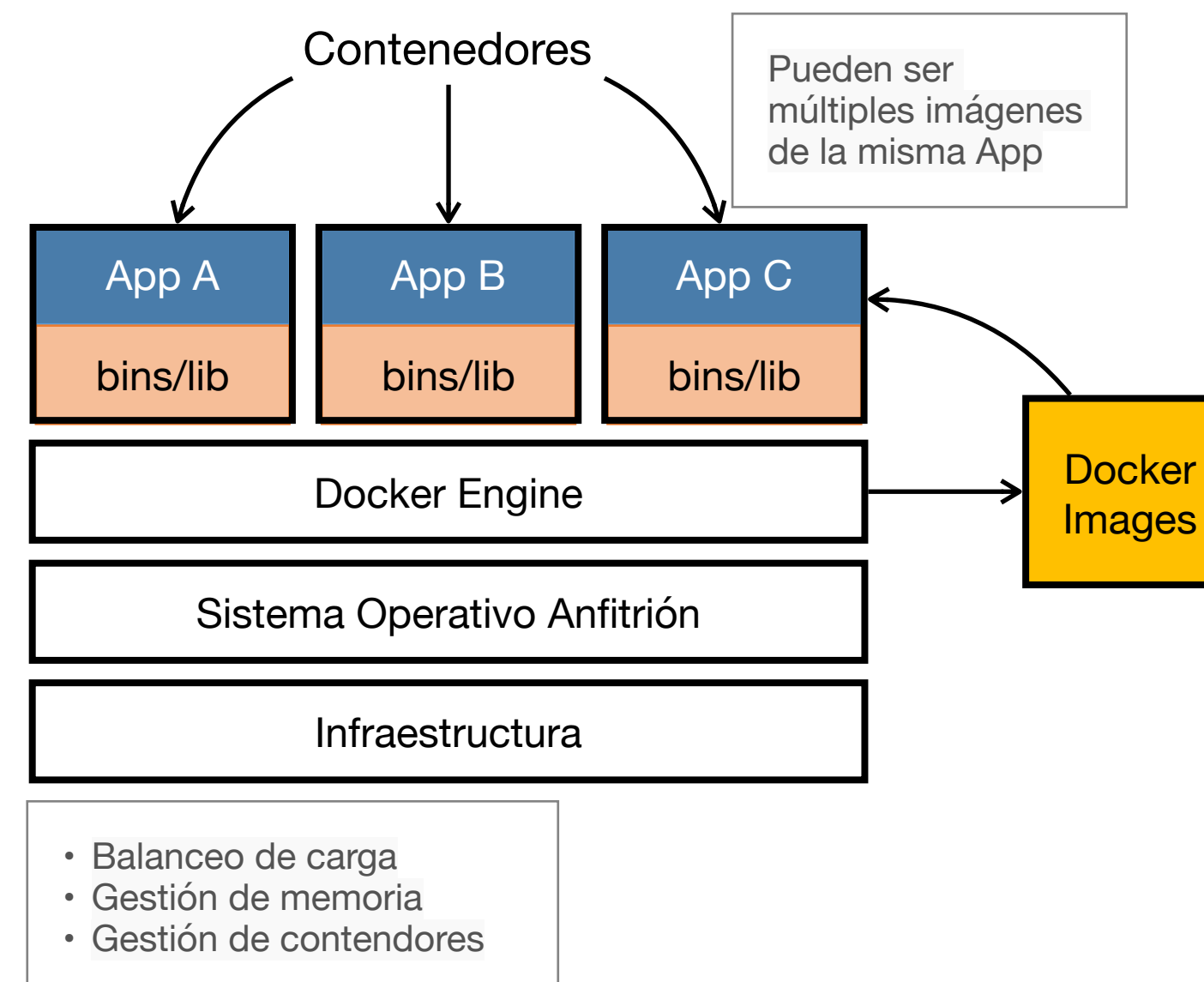
Paso 5 — Reuso y contenerización



- Contenerización del ambiente de desarrollo
- Contenerización de las aplicaciones

- La separación en pequeños componentes permite la reutilización de código.
- Cada componente puede tener una lógica compleja y estar escrito en un lenguaje diferente y requerir un stack de software distinto.
- Es difícil llevar un componente de una máquina o ambiente a otro

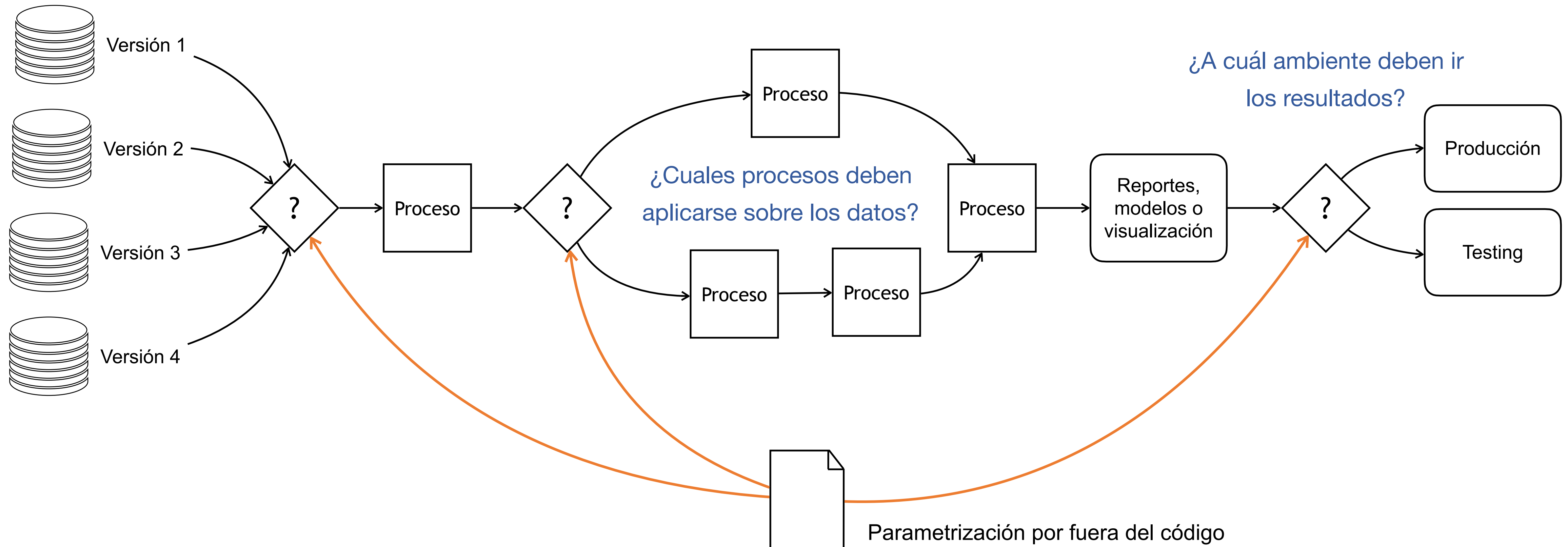
Docker es un proyecto de código abierto para el despliegue de aplicaciones dentro de contenedores para la virtualización de aplicaciones



Pasos para implementar DataOps

Paso 6 — Parametrización del procesamiento

¿Cuál versión de datos debe usarse?



Parametrización por fuera del código

- El pipeline incluye las decisiones en la lógica.
- Permite el ajuste a diferentes condiciones de ejecución.

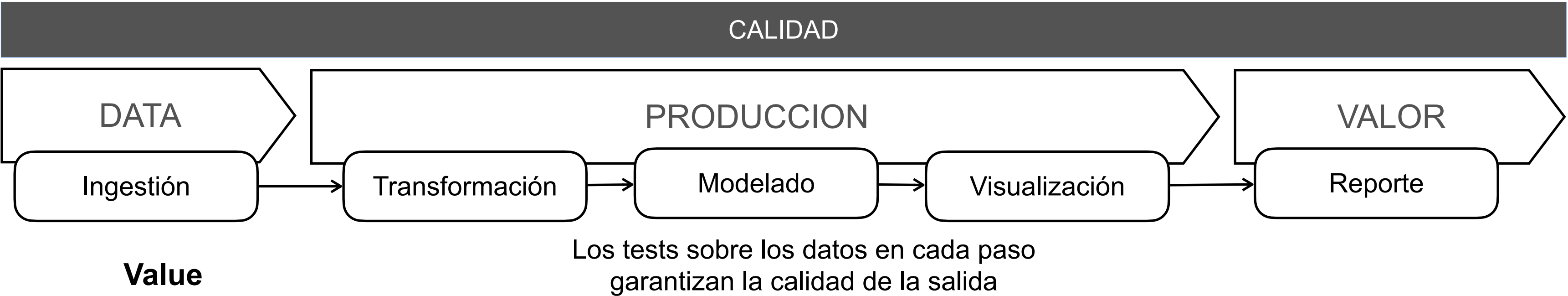
Pasos para implementar DataOps

Paso 7 — Trabajar sin miedo o heroísmo

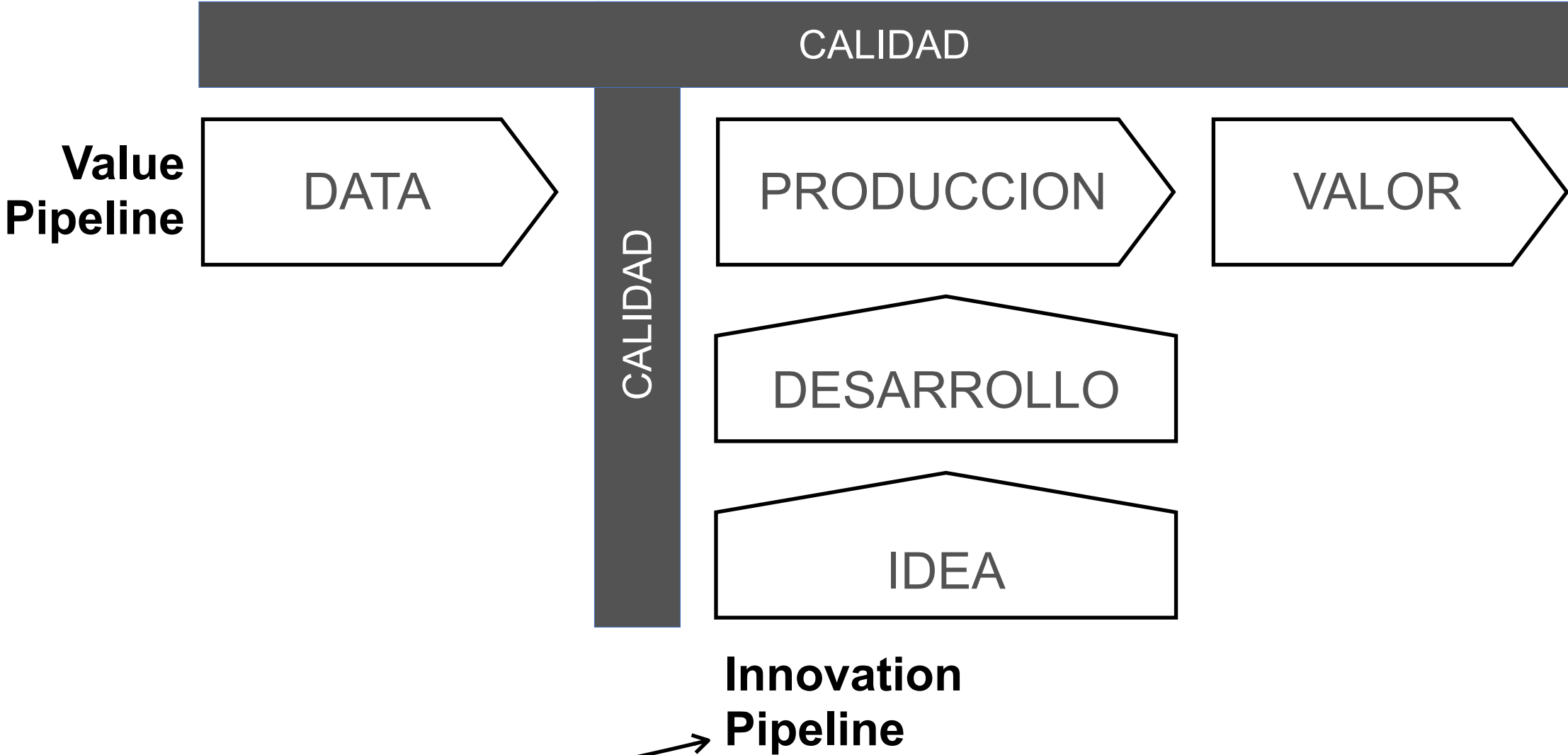
Heroísmo: Trabajo fuera de horario y fines de semana.

- Escenarios desastrosos:**
- Despliegue de cambios que dañan los sistemas productivos.
 - Entrega de datos de poca calidad a los usuarios.

Miedo: Ausencia de confianza en que los resultados sean correctos o los cambios funcionen correctamente.



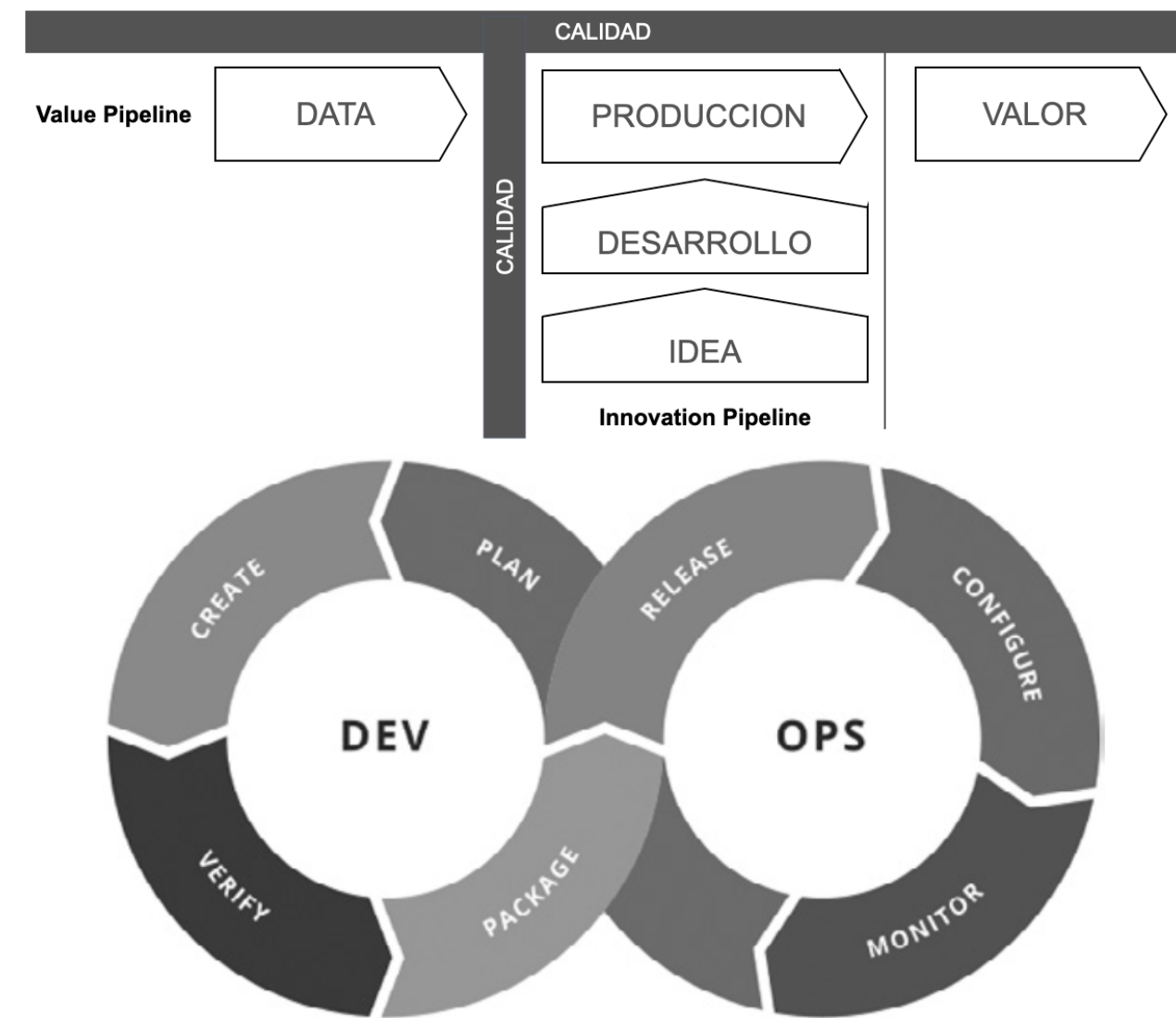
Value Pipeline



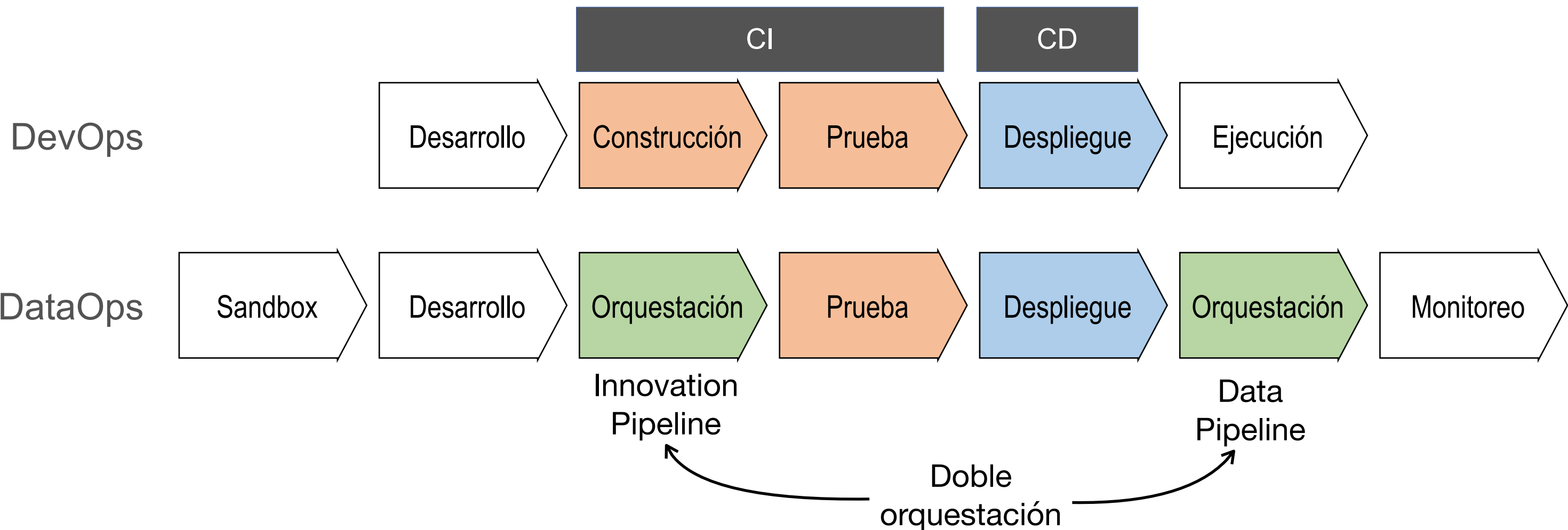
Innovation Pipeline

Diferencias entre DevOps y DataOps

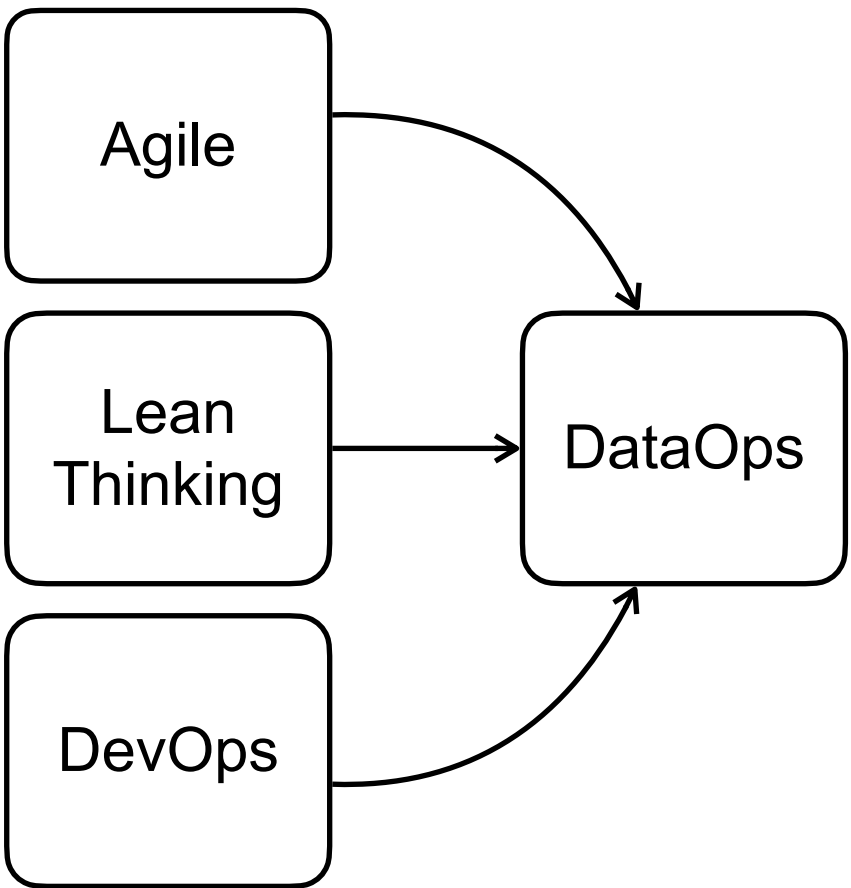
Proceso



Desarrollo y Despliegue



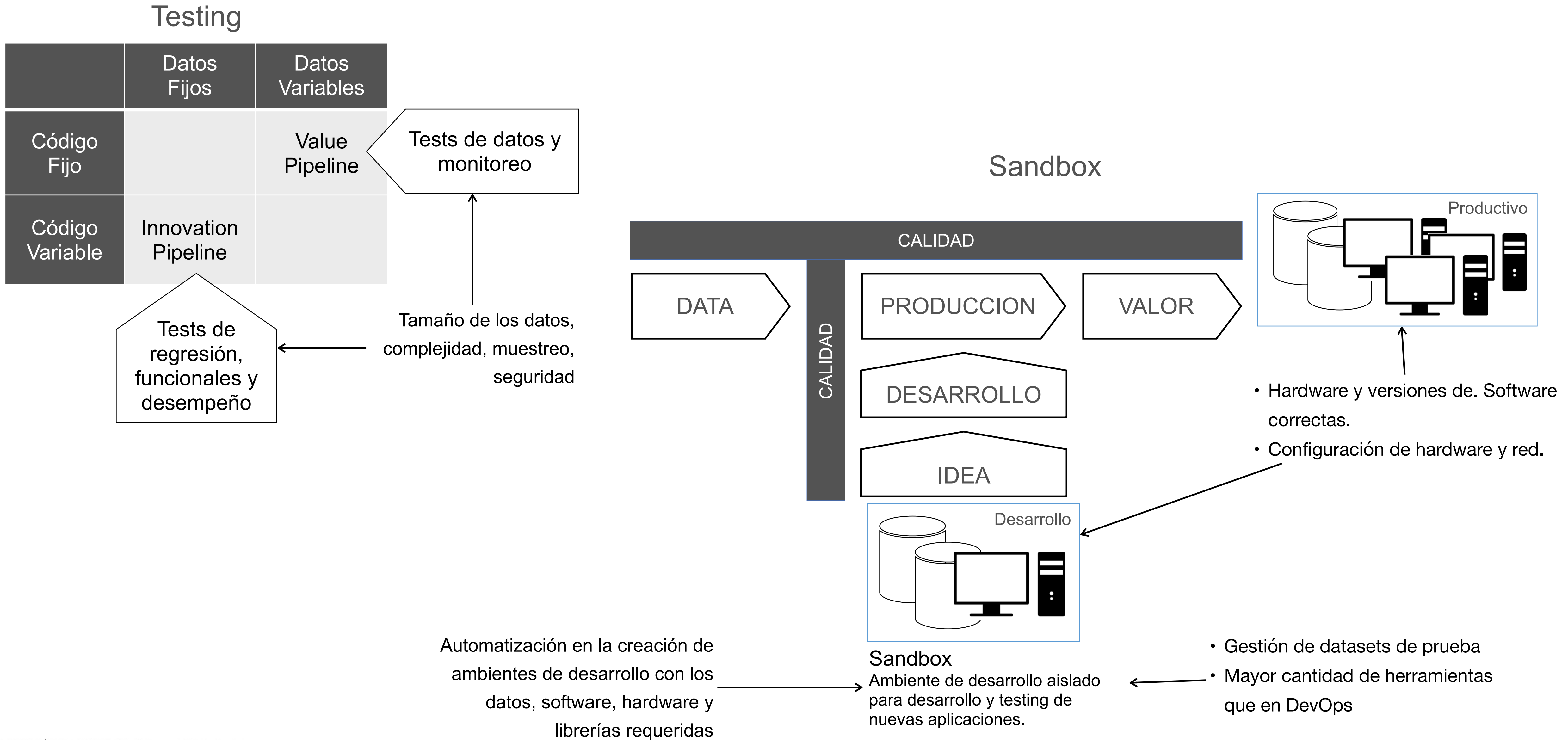
Patrimonio intelectual



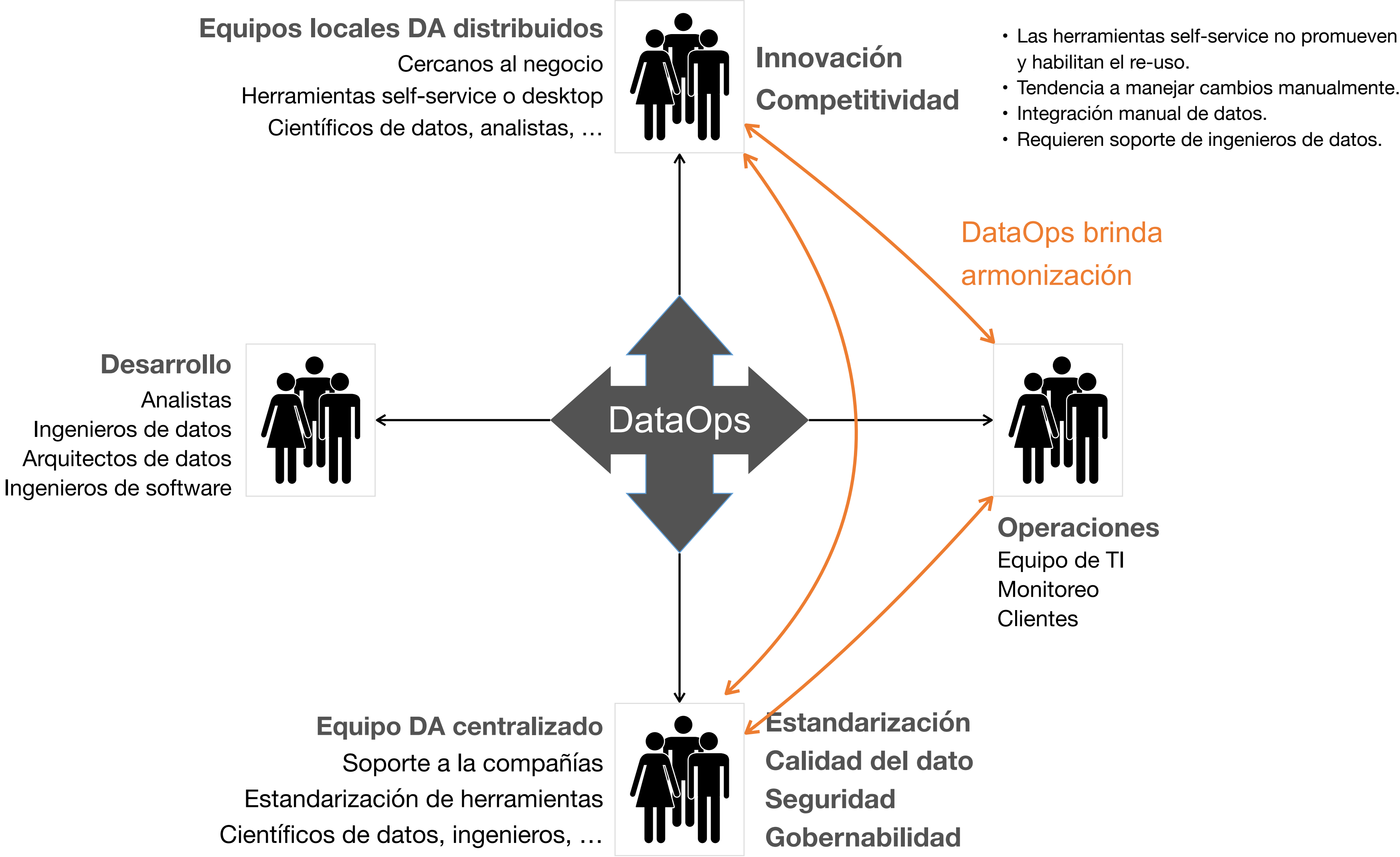
Factores Humanos

DevOps Users & Tools	Ingenieros de software, confortables con código y complejidad de multiples lenguajes herramientas y hardware/software
DataOps Users & Tools	Científicos de datos, ingenieros, analistas que quieren analizar datos y construir modelos. Todo lo demás es complejidad innecesaria

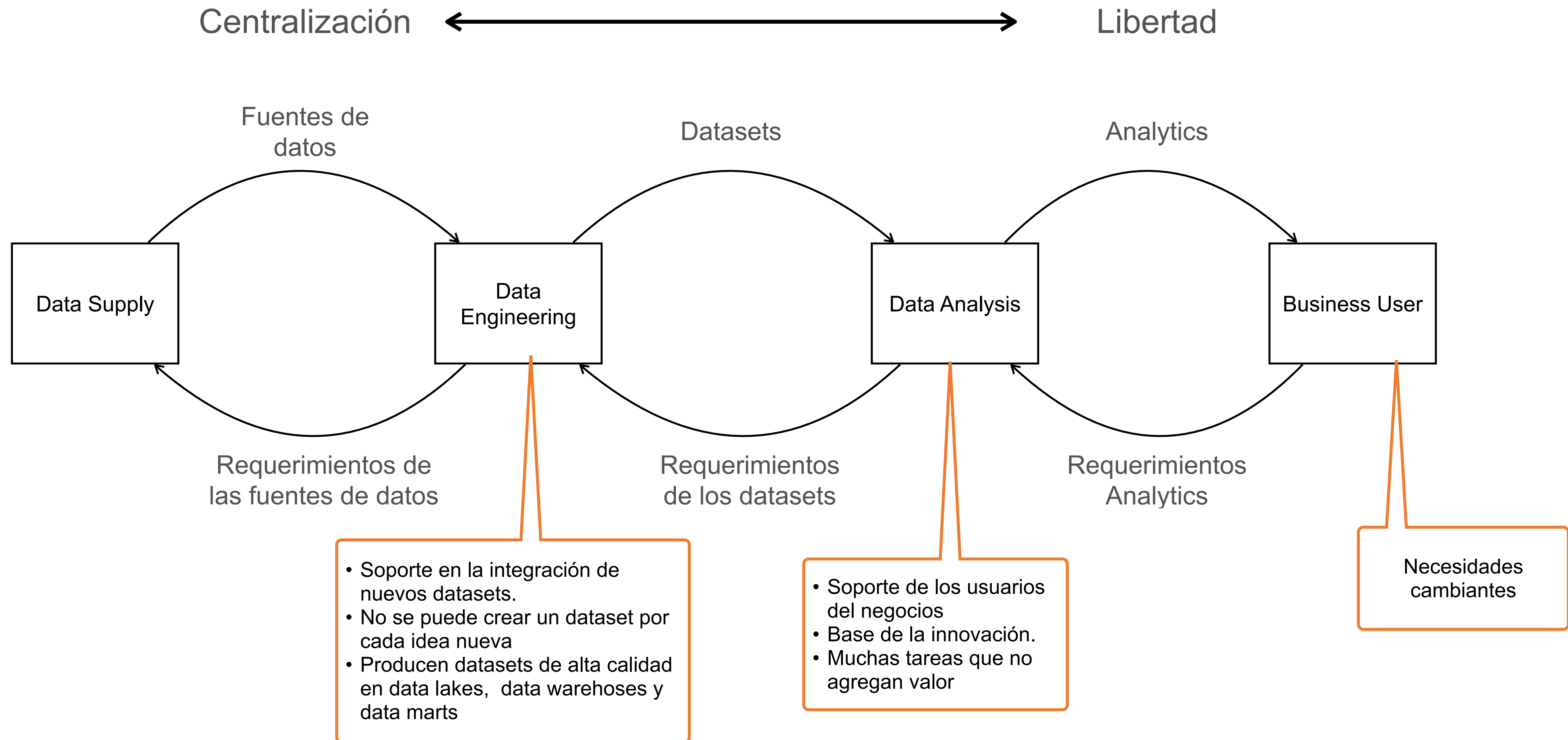
Diferencias entre DevOps y DataOps



Diferencias entre DevOps y DataOps



Cadena de suministro de datos



Implementación de MLOps

Fase 1

- Configuración de ambientes de desarrollo y prueba
- Hardware, almacenamiento y herramientas de software

Configuración de la infraestructura

Desarrollo de modelos

- Desarrollo de modelos en un framework eficiente que permita automatización y optimización
- Desarrollo y manejo de tuberías de datos
- Verificación del desempeño de modelos

Fase 2

Prerequisitos:

- Artifacts auditables con logging
- Modelos documentados y verificados

Transición a Operaciones

Tareas claves:

- Serialización y contenerización de los artifacts
- Model serving
- Despliegue de modelos al ambiente de producción usando CD/CI y pruebas de aceptación
- Cumplimiento de lineamientos de aseguramiento de la calidad

Fase 3

- Monitoreo del desempeño, incidentes y reentrenamiento
- Monitoreo de la telemetría

MLOps

Data Operations

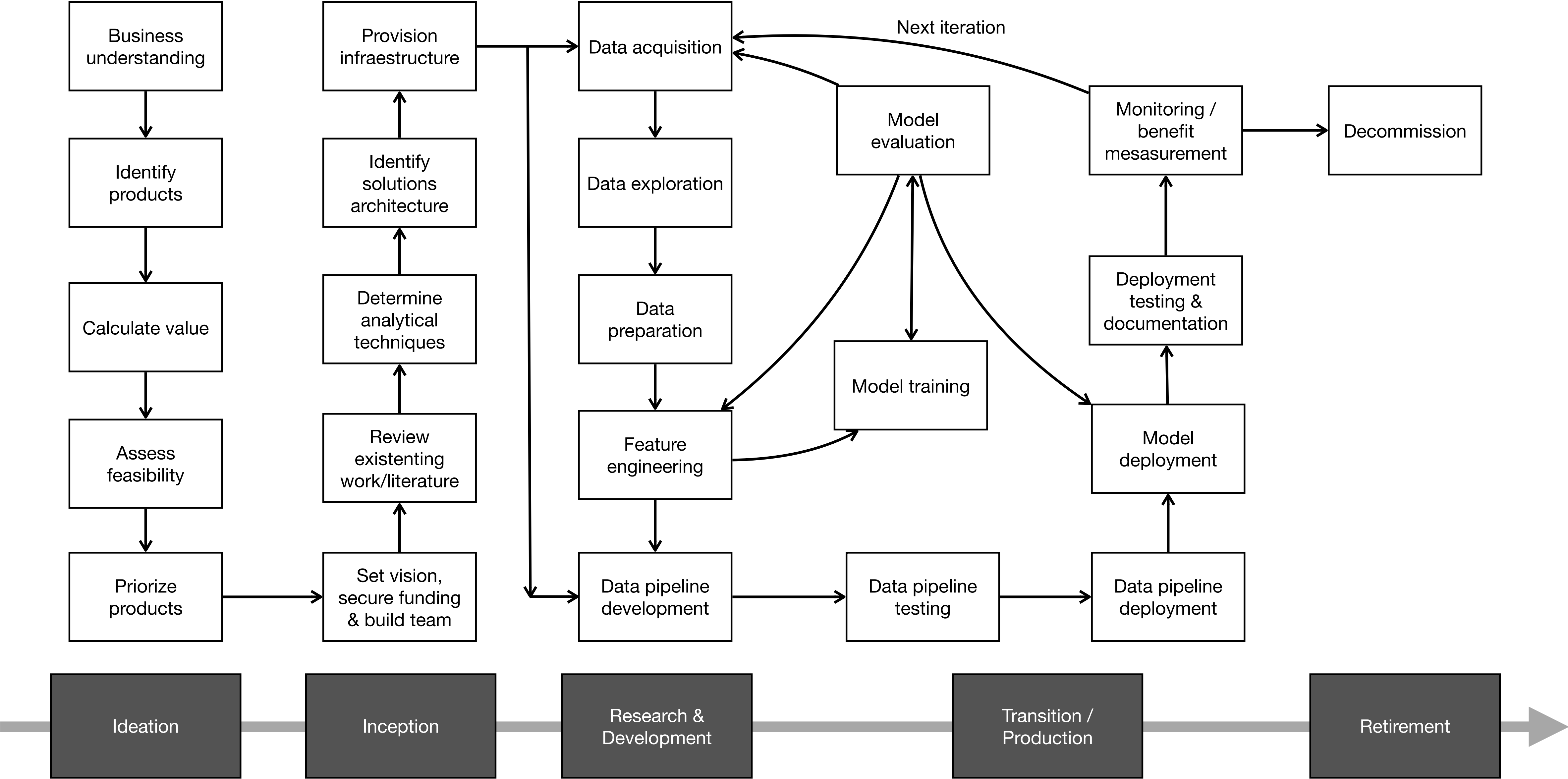
- Monitoreo y solución de incidentes en la tubería de datos y la plataforma de ML, manejo de la seguridad

- Muchos de los principios ágiles para desarrollo de software aplican en DA/Analytics
- En DataOps se debe tener en cuenta que el software por si mismo no hace mejores decisiones
- Una buena decisión requiere datos y algoritmos de alta calidad.
- El desarrollo de productos de datos tiene un ciclo de vida diferente al ciclo de productos de software
- Actividades como entendimiento del negocio, adquisición y limpieza de datos, entrenamiento de modelos, etc son menos predecibles y más iterativas que las actividades típicas en desarrollo de software.

Principios para el manejo del código fuente

- Modularidad
- Funciones dedicadas a una sola tarea
- Estructuración adecuada del código
- Código limpio
- Testing
- Control de versiones
- Logging
- Manejo de errores
- Legibilidad
- Comentarios y discusión

Data Science Lifecycle



Ideation	Feasibility	Analyze	Backlog	Develop	Done
No limit	WIP limited	WIP limited	No limit	Team WIP limits	No limit
• New product & feature creation opportunities • Customer experience enhancements • Business efficiency improvements • Improvements to existing solutions	• Epic hyphotesis statement • Rough sizing of effort • Benefit estimation • Priorititation matrix	• Investigate alternative solutions • Refine cost and benefit estimates • Define MVP • Measurement plan developed • Prioritize	• Epics selected for development of lean portfolio management team • Continuous prioritization of epics	• Epics broken into work teams, stories or tasks. • Teams begin development when they have capacity • Epic tracking	• Benefit measured versus plan • Stop or persist decision made.

Epics: Iniciativas estrategicas que cierran las brecha entre el estado futuro deseado y el estado actual basado en los objetivos de la organización, y que capturan las ideas técnicas y de negocio más significativas. Deben permitir la creación de un MVP.

Epic Hypothesis Statement

Estructure	Description	Example
Achieve	[strategic theme objective]	Compliance goal
Through	[target portfolio vision outcome]	Improved anti-money laundering (AML) risk assessment
For	[customers]	Money Laundering Reporting Officer (MLRO)
By	[proposed solution]	Applying machine learning to automatically monitor customer transactions data to identify anomalies for investigation
Resulting in	[predicted benefit]	Reduced risk of fines
Mesaured by	[metrics]	Value of AML penalties relative to peers
Partnering with	[other teams and partners]	Compliance and IT temas
Integrating	[sources and data types]	Financial transactions and customer identity data
And requiring	[non-functional requirements]	Ability to scale and match transaction volumenenes, and achieve 99.99% availability

¿Qué es DataOps?

JUAN DAVID VELASQUEZ-HENAO, MSc, PhD
Profesor Titular
Departamento de Ciencias de la Computación y la Decisión
Facultad de Minas
Universidad Nacional de Colombia, Sede Medellín

✉ jdvelasq@unal.edu.co
🐦 [@jdvelasquezh](https://twitter.com/jdvelasquezh)
🐙 <https://github.com/jdvelasq>
📄 <https://goo.gl/prkJAq>
📄 <https://goo.gl/vXH8jy>